# Spray Computers: Explorations in Self-Organization

Franco Zambonelli[1], Marie-Pierre Gleizes[2], Marco Mamei[1], Robert Tolksdorf[3]

[1] *DISMI - Università di Modena e Reggio Emilia – Reggio Emilia – Italy*
[2] *IRIT – Universitè Paul Sebatier – Toulouse – France*
[3] *Institut für Informatik – Freie Universität of Berlin – Berlin - Germany*

## Abstract

*We envision a future in which clouds of microcomputers can be sprayed in an environment to provide, by spontaneously networking with each other, an endlessly range of futuristic applications. However, beside the vision, spraying may also act as a powerful metaphor for a range of other scenarios that are already under formation, from ad-hoc networks of embedded and mobile devices to worldwide distributed computing. After having detailed the different spray computers scenarios and their applications, this paper discusses the issues related to the design and development of spray computer applications, issues which call for novel autonomic approaches exploiting self-organization as first-class tools. Finally, this paper presents the key research efforts being taken in the area and attempt at defining a rough research agenda.*

***Keywords****: Spray Computers, Bottom-up Software Engineering, Self-organization.*

## 1 Introduction

With the MEMS revolution in full swing, micro-sensors are following manufacturing curves that are at least related to Moore's Law [Pis00]. This trend, when combined with both the push for low power communication and computation devices and for the ubiquitous provisioning of data and services, pave the way for the spray computers revolution.

It is not hard to envision a future in which network of micro computers will be literally sold as spray cans, to be sprayed in an environment or on specific artifacts to enrich them with functionalities that, as of today, may appear futuristic and visionary [Nag03, ZamM02, Pis03]. The number of potential applications of the scenario is endless, ranging from smart and invisible clothes, intelligent interactive environments, self-assembly materials and self-repairing artifacts.

However, the vision of spray computers may also act as a powerful metaphor for a range of other scenarios that are already under formation. These include distributed applications in embedded, possibly mobile, ad hoc networks [MamZL04, Pat03, Phi04], as well as distributed service- and data-oriented activities on the Internet [RowD01, Rat01]. In fact, besides the different physical scale of the components involved and of their interactions (from micro-computers interacting within networks extending across a few meters, to Internet hosts interacting at a world-wide scale), all of these types of spray computer networks raise similar challenges as far as development and deployment of applications is involved, calling for radically novel – "autonomic" [Ten00, KepC03] – approaches to distributed systems development and management.

On the one hand, to avoid the unaffordable efforts related to the placement, configuration, and maintenance of such systems, there is the need of approaches enabling of deploying components without any a priori layout effort, and letting components to self-organize their application activities and self-retune their overall behavior depending on specific contingencies (e.g., localized faults and environmental changes) [KepC03]. On the other hand, the autonomous and decentralized nature of the activities in such scenarios, together with the possibly unpredictable dynamics of the operating environments, is likely to make those systems exhibit spontaneously "emergent" behaviors - as recent observations in several types of decentralized networks (i.e., the Internet, the Web, as well as Gnutella) suggest. Therefore, there is also need for methodologies to predict and control the emergence of such behaviors and, when possible, offensively exploit them for the achievement of complex distributed tasks [BerGPP02, ParBS02, MamRZ05].

This paper aims at exploring the above issues and is organized as follow: Section 2 details our vision about spray computers, starting from the micro-scale (i.e., literally spray-able computers), to the medium scale (smart artifacts and MANETs), up to the macro-scale (wide-area networks). Section 3 will present and discuss the major issues arising when exploiting self-organization for the design and development of

applications for spray computers. Section 4 will present a survey of some representative research efforts being taken in this area, discussing their advantages and drawbacks. Section 5 concludes the paper by attempting to define a roadmap of research activities in the area of spray computers.

## 2 Spray Computers and Applications

The concept of spray computers will soon pervade the ICT scenarios at every scale and at every level. In the following we will briefly survey our idea of future computer-based systems from the micro-scale (literally spray computers), to the medium-scale (handheld and wearable computers) to the global scale (Internet and Web computing).

### 2.1 The Micro Scale

As proved in the context of the Smart Dust project at Berkeley [BerG97, Pis00], it is already possible to produce fully-fledged computer-based systems of a few $mm^3$, and even much smaller ones will be produced in the next few years. Such computers, which can be enriched with communication capabilities (radio or optical), local sensing (e.g., optical, thermal, or inertial) and local effecting (e.g., optical and mechanical) capabilities, are the basic ingredients of our spray computers vision.

Spray computers, as we imagine them, are clouds of sub-millimeter-scale microcomputers, to be deployed in an environment or onto specific artifacts via a spraying or a painting process. Once deployed, such components will spontaneously network with each other and will coordinate their actions (i.e., local sensing and effecting) to provide specific "smart" functionalities. We imagine it will be possible, say in 2020, to go to the local store and there buy, for a few euros, a "pipe repairing" spray, made up of a cloud of MEMS microcomputers capable of navigating in a pipeline, recognizing the presence of holes, and self-assembling with each other so as perfectly repair the pipe. Similarly, we could imagine a spray to transform our everyday desk into an active one, capable of recognizing the positions and characteristics of objects placed on it and letting them meaningfully interact.

Another peculiar application we envision is the "spray of invisibility" (described in [ZamM02]): a spray of micro devices capable of receiving and re-transmitting light emissions in a directional way, and capable of interacting with each other via short-range wireless communications. When an object is covered by a layer of such spray, the emissions of the devices make external observers perceive exactly the same light configurations that they would have perceived if there were nothing in between. In fact sensors on the rear side of the object can receive such configurations and, via distributed coordination, can communicate them to emitters on the observer's side to be retransmitted. Other types of application one could envision include any type of self-assembly artifact [Nag03, StoN04], there included thing like Terminator T-1000, the nano-swarms of Michael Crichton's novel "Prey" [Cri02], and MEMS-based artificial immune systems and drugs [Pis03].

Whatever the applications one envisions, the key characteristics that will distinguish spray computers applications from traditional distributed computing systems are not – as one could at first think – the scale at which processes take place. After all the fact that a process executed on a micro device rather than on a high-end computer does not change it basic nature. Instead, what we think strongly distinguish spray computers are the facts that:

- computational activities take place in a network whose structure derives from an almost random deployment process (as a spraying process is), and that is likely to change over time with unpredictable dynamics (due to environmental contingencies, failure of components, or simply mobility);
- the number of (hardware and, consequently, software) components involved in a distributed application is dramatically high and hardly controllable. It is neither possible to enforce a strict configuration of software components nor to control their behavior during execution at a fine-grained level.

Both the above characteristics – which are somewhat exhibited by current sensor networks technologies [Est02] – compulsory call for execution models in which applications are made capable of self-configuring and self-tuning their activities in a spontaneous and unsupervised way, adapting to whatever network structure and surviving network dynamics. In other words, the spray computer vision calls for the achievement of the autonomic computing vision [KepC03].

### 2.2 The Medium Scale

Spray computing, other than a likely incoming technology, can also act as a power metaphor for describing the key characteristics of the emerging scenarios of ubiquitous and pervasive computing, as enabled by handheld, wearable, and embedded networked computing systems.

We already typically carry on two or three computers (i.e., a cell phone, a laptop, and possibly a PDA). Also, our houses are already populated by a variety of microprocessor based furniture (e.g. TVs, phones, etc.). However, at the moment, the networking capabilities of these computer-based systems are under-exploited. On

the contrary, very soon, the world around us will be densely populated by personal-area networks (e.g., the ensemble of Bluetooth enabled interacting computer-based devices we could carry on or we could find in our cars), local ad-hoc networks of handheld computers (e.g., networks of interacting PDAs carried by a team that have to directly interact and coordinate with each other in an open space), and furniture networks (e.g., Web-enabled fridges and ovens able to interact with each other and effectively support our cooking activities in a coordinated way).

What we want to emphasize here is that the above types of networks, although being formed by different types of computer-based devices (let's say, medium-end computers) and at different physical scales than literally spray computers, shares with them the same issues as far as the development and management of distributed applications is concerned. In fact:

- most of these networks will be wireless, with structures dynamically varying depending on the relatives positions of devices, all of which intrinsically mobile (the persons in an ad-hoc network can move around in an environment and the position of home furniture can changed on needs) and characterized by the dynamic arrival dismissing of nodes (a PDA running out of power or a new home furniture being bought).
- even if technically possible, it is simply not commercially and economically viable to consider deploying applications that would require explicit configuration and explicit tuning to meet the amorphous and dynamic nature of the networks in which applications will be expected to operate.

Also in these cases, new approaches are needed to develop applications as if they were to execute on a network of spray computers.

### 2.3 The Global Scale

Similar considerations can apply to the case of macro-scale networks made up of high-end computer systems, i.e., the Internet and the Web. In fact, the dramatic growth of these networks and of the information and traffic to be managed over them, together with the increasing request for ubiquitous connectivity and the peculiar structures exhibited by such networks [AlbJB00, RipIF02] – making them sorts of "spray computers" networks – have raised researchers' attention to the need of novel approaches to distributed systems management.

Traditional approaches to management, requiring human configuration efforts and supervision, fall short when the number of nodes in the network (or the number of interrelated services and links in the Web) grows in a fully decentralized way, and when the presence of the nodes in a network is of an intrinsically ephemeral nature, as it is the case of laptops and, with regard to the Web, of several non-commercial data and services.

In a number of application scenarios, the need to access data and services according to a variety of patterns and independently of the availability/location of specific servers and of the dynamics of the network have suggested the adoption of P2P approaches [Rat01, RipIF02]. In P2P computing, instead of promoting a strict control over the execution of software components and of their interactions (an almost impossible task given the dynamics of the scenario), the idea is to promote and support adaptive self-organization and maintenance of a structured network of logical relationships among components (i.e., an overlay network), to abstract from the physical "sprayed" nature of the actual network and survive events such the arrival of new nodes or the dismissing of some nodes.

Overlay networks are currently the most widely investigated approach to distributed application development and management in worldwide computing, and are leveraging a variety of useful applications facilitating access to (and coordination over) a variety of world-wide distributed data and services. Although self-organizing overlay networks – as they are studied today – are not necessarily the only and best approach, the attention towards them is the body of evidence of the need – also in this scenario – for new self-organizing approaches to distributed application development.

As a final note, we emphasize that, although the micro, medium, and global scale currently represent almost separated worlds, this will not be the case in the near future. All the above systems will probably be in the near future part of a mega decentralized network, including traditional Internet nodes, smart computer-enriched objects and furniture, networks of embedded and dispersed micro-sensors. For instance, the IPv6 addressing scheme will make it possibly to assign an Internet address to every cubic millimeter in the earth surface [ImiG00], thus opening the possibility for each and every computer-based component to become part of a single worldwide network.

## 3 Programming Spray Computers

Programming a spray computer (i.e. developing applications to be deployed on a network of spray computers) means to engineer a pre-specified, coherent and useful behavior from the cooperation of an immense number of unreliable parts interconnected in unknown, irregular, and time-varying ways. This translates in devising algorithms and control methodologies to let the sprayed computing devices self-organize their interaction patterns and their activities: devices have to start

working together without the presence of any a-priori global supervisor or centralized controller.

The basic low-level mechanism upon which to rely to enable self-organization appears quite well-understood and are basically the same whatever the scale, whether that of sensor networks or that of wide-area P2P computing. Among the others: dynamic discovery of potential communication partners and of available services via broadcasting [HigB01]; localization and navigation in some sorts of spatial environment, whether physical (as in sensor networks) or computational (as in P2P systems) [ZamM05].

What is still missing is an assessed understanding of how to design, develop, and manage, self-organizing applications for these kinds of systems, leading to some general purpose methodologies and programming environments. The main conceptual difficulty being that, while spray computers enable a direct-engineered control only on components' local activities, a variety of diverse application goals have to be achieved at a global scale.

Identifying some general and abstract solution to enable the design and development – via a proper programming of self-organizing activities – of specific global application goals, would have a dramatic impact in all sketched scenarios (micro, medium and global scale). In this paper, without having solutions at hand, we try to identify some key directions to investigate. In particular, the following of this section sketches four different approaches, and support the presentation with the help of two simple, yet representative, case studies.

### 3.1 Direct Engineering of Self Organization

**Overview.** Direct engineering approaches to self-organization basically aims at defining distributed algorithms that, starting from a few basic mechanisms (e.g., broadcast and localization), and exploiting local interactions and local computations, can provably lead a system (or parts of it) to a final coherent global state.

Unlike traditional distributed algorithms, self-organizing algorithms disregard micro-level issues such as ordering of events, process synchronization, and structure of the underlying networks (issues for which no possibility of control is assumed). Rather, they focus on the fact that the algorithm will eventually converge despite micro-level contingencies and that it will keep the system in the stable state despite perturbations (e.g., changes in the network structure).

**Examples.** A typical example of a direct engineering approach to self-organization is distributed self-localization. There, a number of randomly distributed particles can determine their geographical position starting from a few "beacon" particles, possibly self-determined via leader election and acting as reference frame. By recursively applying a local triangulation mechanism to have each particle determine their position w.r.t, to close particles, a global coherent positioning of all particles in the reference frame is eventually reached [Nag02], even in a large network, and whatever the specific pattern according to which the process propagate in the whole network.

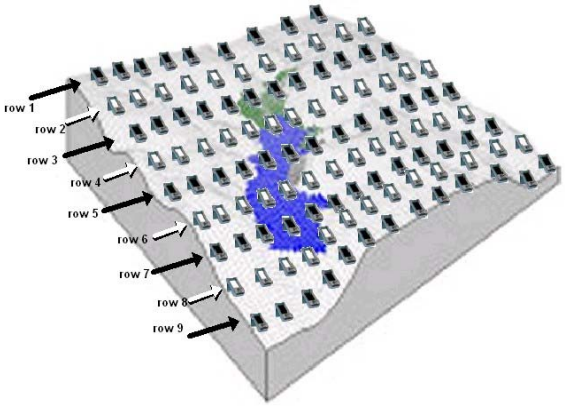



**Figure 1: Direct self-organization of activity patterns in a grid of sensors deployed on a landscape**

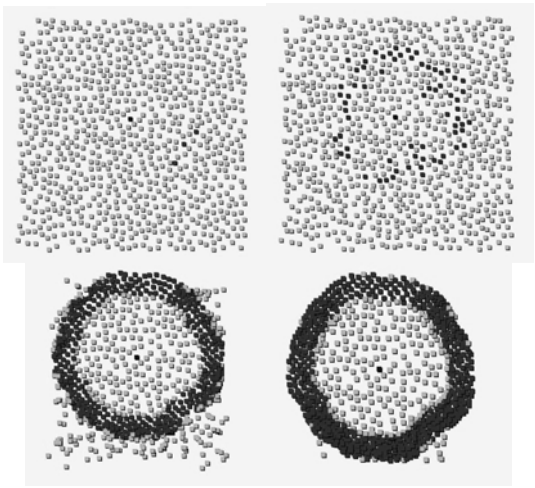


**Figure 2: Sequences of direct self-organization of a swarm of cooperative mobile robots into a circular shape**

Self-localization algorithms turn out to be very useful in a variety of scenarios, and specifically in sensor networks [Est02]. In general, the very goal of a sensor network is to spatially coordinate the activities of the sensors to cooperatively achieve specific sensing activities in an environment. Thus, in a sensor network, once each sensor has self-localized itself accordingly to some absolute or relative reference frame, sensors can easily self-organize their activities according to that. For

example, they decide to alternatively get to "sleepy" state according to some pre-configured geographical rule, i.e., in strip-like patterns, so as to save battery power without undermining sensor coverage (Figure 1). Being more visionary, a very similar approach can be exploited for the production of smart paints.

Another examples that may be of use to sketch a different "direct" approach to self-organization relate to the problem of having a swarm of simple mobile robots be programmed to coordinate their respective movements and create variety of global shapes. Apart from the vision of computational self-assembly [Nag02, StoN04], the problem has also more practical short-term applications: coordinating the movements of navigator-equipped cars, coordinating the movements of a rescue team provided with PDA [MamZL04]; enforcing self-deployment of sensor networks and of complex robots in a landscape [MCL04]. In particular, our goal is to devise distributed algorithms that, by locally driving the movements of the particles, eventually lead the system to self-organize in globally regular shapes.

Given a number of particles distributed in an environment, it is possible to devise distributed algorithms that, by locally driving the movements of the particles, eventually lead the system to self-organize in globally regular shapes [MamVZ04]. For instance (see Figure 2): a simple leader election algorithm can be exploited to have particles self-determine their center of gravity. Then, the center of gravity can propagate in the network with a hop-increasing data structure (a sort of "gravitational field"), attracting all other particles toward the center, and until a specific distance from the center is reached. The result is in an almost circular organization of particles.

**Discussion.** Direct engineering approaches to self-organization have the great advantage of enabling engineers to achieve "by design" a specific robust self-organized behavior. Engineers, following standard methodologies, can identify design distributed algorithms to control distributed components to let them behave as specified by the application.

Unfortunately, such approaches are effective only for a limited number of application needs. In fact, with direct approaches, only very simple global states of equilibrium (or, which is the same, only very regular patterns of activity) can be enforced, i.e., all those that can be modeled in simple linear terms. Engineering more complex behavior involving non-linear phenomena such as differentiation in activities, navigation and localization in complex manifolds, enforcement of complex coordination patterns, requires facing very higher complexities, and the complexity of the distributed algorithms involved may become overwhelming. Most importantly, it may require a priori assumptions on the configuration of the system that would limit its self-organizing nature and, consequently, its degree of adaptivity and its robustness.

With regard to the sensor network example, even the achievement of the simple strip-like patterns in Figure 1 requires the prior execution of the self-localization algorithm and some a-priori agreement on geographical rules. With regard to the mobile particles example, making particles self-organize into complex non-symmetrical patterns by direct self-organization would require either a very complex algorithm to be executed by the particles, or having particles provided with some a priori information on where to go, that is, some a-priori configuration efforts.

For all these cases, a reverse engineering approach may be preferred.

### 3.2 Reverse Engineering of Self Organization

**Overview**. Reverse engineering approaches to self-organization aims at achieving complex coordinated behaviors by recreating in spray computers (and by adapting to specific application needs) the conditions to make some emergent coordinated behaviors observed in other systems and in nature emerge in the computational spray computer system too. In these cases, due to the complexity (and often non-linearity) of the phenomena involved, engineers have no direct control on the evolution of the system, nor they can somehow prove that the system will behave as needed. Simply, they can be reasonably (i.e., probabilistically) confident that the global evolution of the system will eventually lead to the desired globally coordinated behavior.

Clearly, simulations will be the workhorse of reverse engineering approaches [EdmB04]. Simulations of spray computer systems will not only provide a framework on which to test the functionalities of a systems once developed, but they will be an integral part of the design and development process. Since the behavior of the components and of their interactions that can lead to the desired global behavior can hardly be modeled and predicted "on paper", simulations appear to be the only tool with which to have feedback on how a system will actually work. In other words, in reverse engineering approaches, the modeling phase consists in verifying via simulations the correctness of an idealized model suitable, but not necessarily close, to the target scenario (this model can be for example a biological or social model), then to refine the model and the simulations (that also realize a prototype implementation of the model) to rend both enough similar to the actual scenario to be taken in consideration as a candidate solution.

**Examples.** In the past few years, several approaches to self-organization relying on the reverse engineering of diverse natural phenomena have been proposed in

different areas and have shown their effectiveness in achieving difficult global coordination tasks. For instance, the phenomena of ant foraging [BabMM02, MenT03] and gossiping [BraE02] turn out to be useful to discover path to information and diffuse information in networks of spray computers.

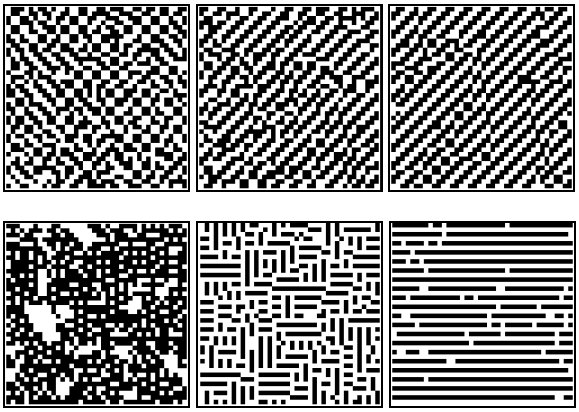


**Figure 3. From left to right, the evolution of two different types of stochastic cellular automata, evolving into macro-level spatial patterns of coordination**

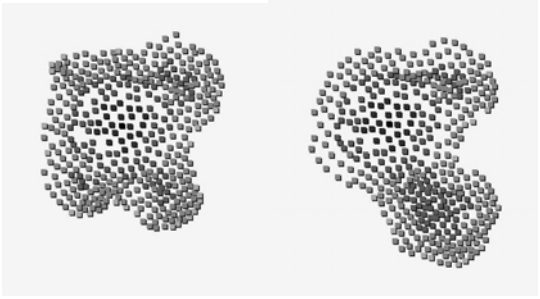


**Figure 4. Emergence of non-symmetrical patterns in swarms of simple cooperative mobile robots**

Getting back to our sensor network example, one can get inspiration to some behaviors exhibited by cellular automata. Cellular automata are a model for locally interacting cells, each with a local state (finite-state-automata) that get updated on the basis of simple rules and accounting the state of neighbor particles in a grid [Wol02]. Cellular automata, on the basis of simple rules and with a moderated injection of stochastic behavior, exhibit the emergence of macro-level spatial patterns of coordinated activity, e.g., among the others, a variety of strip-like patterns (Figure 3). Reverse engineering and reproducing a similar behavior in a grid of sensors (and, more generally, in any – even amorphous – network of sensor) is dramatically simple. Basically, this amount at having the activity state of a sensor (i.e., sleepy versus active state) be decided on the basis of rules that account for the state of neighbor sensors, mimicking the behavior of a cellular automata [MamRZ05]. A variety of other types of distribution of activities can be analogously enforced, also by extending from cellular automata to more general phenomena of spatial differentiation of activities [ShiM02].

With regard to the example of the swarm of mobile robots, specific application needs may require robot to assume a specific non-symmetric form. In particular, it is possible to enforce the emergence of non-symmetrical patterns by getting inspiration from the biological formation of morphogenesis. There, it can be observed that differentiation from regular symmetrical patterns occur due to the contrasting forces induced by cells increasing in number and still have to adhere to each other in a limited physical space. Similarly, in mobile computational particles, one can impose a constraint on the average density of cell, making cells repel each other if a too high density is reached. This constraint, contrasting with the gravitational force field that would attract them toward the center of gravity, force them organizing in non-symmetrical patterns (see Figure 4).

**Discussion.** Reverse engineering approaches to self-organization have several advantages. First, it is possible to rely on results from other disciplines to explore a variety of complex coordination phenomena to be exploited in spray computers systems. Second, once the basic mechanisms underlying a self-organized behavior are understood and properly reproduced via simulation, programming an actual system to exhibit such behavior is dramatically simple, and it reduces to programming typically simple local rules and local interactions. In addition, the resulting system is intrinsically robust and adaptive. The sensor network examples clearly shows this: strip-like patterns of activity can be obtained in a very simple way, without requiring the prior execution of a complex and costly self-localization mechanism, and without requiring any a priori agreement on geographical rules for selecting the sleepy/active state in sensors.

Unfortunately, reverse-engineering approaches incur in two main potential drawbacks. In fact, the evolution of a system relying on complex reverse engineered phenomena – typically involving non-linearity – may cause several potential final states to be reached by the system, each of which potentially stable, and without the possibility of predicting which ones will be actually reached after the self-organization process. In some cases, all of these states may be equivalent from the application viewpoint (e.g., in ant foraging, what matters is that a reasonably short path to food/information is reached, no matter what the path actually is). Also, in these cases, the presence of multi stable states may be also advantageous, because this ensure that the system,

even if strongly perturbed (e.g., due to network or environmental dynamics), will be able to soon re-organize its activity into another stable state. However, in several other cases, the designer may wish that its system self-organizes to a specific global state, not to any one.

For instance, in the case of the sensor network examples, when applying the cellular automata approach, one may wish that the network self-organize into "horizontal" rather than "vertical" stripes (cfr. Figure 3). In the case of cooperative mobile robots, specific application needs of self-assembly or of landscape, may require robot to assume a specific non-symmetric form, not any of the ones that could emerge from the self-organization process. For both cases, the same problem applies in the case of systems perturbation: once an already self-organized system gets perturbed, the designer may not wish it to re-stabilize to a different state.

The above problems, can be somewhat solved by trying to mix together direct and reverse engineering approaches.

### 3.3 Control of Self-Organization

**Overview.** When the evolution of a system can lead to several final global states, and only a limited set of these are useful to the specific application purposes, the problem arise on how to control/direct the spontaneous evolution of the system so as to ensure that it will self-organize as desired.

To enforce such control one can think at:

- Using a *reverse engineering* approach to exploit a phenomena – and the associated low-level mechanism – that can lead to a useful global behavior;
- Coupling it with some *direct engineering* to somewhat control the evolution of the system global behavior;

In general, such a mixed approach presents a number of challenges. On the one hand, introducing some sorts of direct engineered control should be done without undermining the basic advantages of the reverse engineering approach, i.e., its capability to promote the spontaneous formation of complex and robust patterns of activity with little design and coding efforts. On the other hand, for such a mixed approach to be possible, it is necessary that both direct algorithms and reversed self-organizing mechanism are modeled and coded with the same set of basic abstractions.

**Examples.** We can still refer to the two already discussed examples to clarify what such a mixed approach for the control of emergent behaviors could be.

In the example of sensor networks, we have verified via cellular automata simulations [MamRZ05] that it is indeed possible to control the evolution of the system so that its evolution will end in a specific global pattern – among several possible ones. This control can be achieved by having a limited percentage of the cells in the cellular automata (i.e., a limited number of sensors in the network) modified so as to act on more constrained rules than those of the other cells. This limited percentage of cells is nevertheless able to globally influence the spontaneous evolution of the system so as to direct it towards a specific final configuration. With this methodology, we have also been able to have a system evolve towards configurations that would have been (probabilistically) very hard to achieve via spontaneous evolution only (Figure 5).

In the example of the mobile robot swarm, we have successfully enforced some sort of distributed control of emergent behavior, by properly mixing the phenomena making non-symmetrical patterns emerge (described in the previous section) and sort of leader election algorithm. In particular, by using specific computational fields we have been able to let the system self-select a specific number of particles – equidistant from each other and from the center. These particles can then start to act as if they had a higher gravitational mass (i.e., diffusing a gravitational field of higher value), which makes it possible to control the emergence of polygon-like structures (see figure 6) with any required number of vertices. Overall, the solution preserves enough simplicity and robustness.

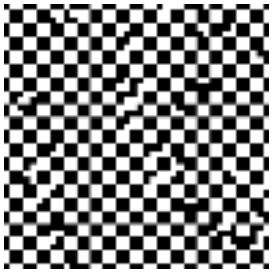


**Figure 5. Control of emergent behaviors in cellular automata**

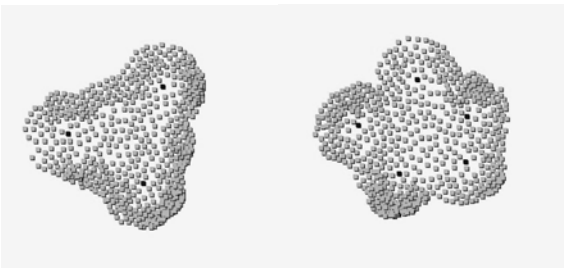


**Figure 6. Controlling the emergence of specific non-symmetrical patterns in swarms of mobile**

**cooperative robots**

**Discussion.** Besides the above two examples, and a few additional ones described in the next section, the general problem of combining self-organizing emergent behaviors and direct engineering approaches to obtain a reliable, adaptive, robust self-organizing system is still open an widely uninvestigated. Nevertheless, to the authors' opinions, this will represent one of the key challenges for the whole research area of autonomic and self-organizing computing.

The urge for appropriate control models and for a uniform approach to direct and reverse engineering of self-organization appears even more compulsory when considering another factor: in several cases, even simple systems engineered with a direct approach may – due to simplifications in their modeling – exhibit unexpected self-organizing behaviors. Although sometimes such unexpected behaviors may be irrelevant or even useful for them to be offensively exploited (consider, e.g., the emergence of scaling in complex networks, and the advantages it carries to the robustness of the network [AlbJB00]), sometimes they may be damaging and introduce the need to defend from them by proper forms of control.

# 4 Relevant Research Projects

Several projects around the world are starting to recognize the above needs and are facing issues related, to different extent, to the engineering and programming of spray computers systems and applications. Without the ambition to be exhaustive, we present here a few relevant threads of activities and discuss their shortcomings.

## 4.1 Micro-scale

Most of the researches in the area of micro-scale spray computers (i.e., literally spray computers) are performed in the context of the "sensor networks" research community [Est02]. There, the key issues being investigated relate to the identification of effective algorithms and tools to perform distributed monitoring activities by a cloud of distributed sensors in a physical environment (tracing the position and movement of an object, determining the occurrence of specific conditions, reporting sensed data back in an efficient way). These researches are indeed providing good insights on the theme of self-organization and are leading to some very interesting results. Techniques for self-localization, self-synchronization of activities, adaptive data distribution, all of which of primary importance for any type of spray computers, have been widely investigated [Est02, NagBS03]. Still, we feel these researches are somewhat limited by several main factors. First, most approaches rely on direct self-organization, while reverse engineering approach are only occasionally exploited. Second, the accent on "sensing" tends to disregard the "actuating" factor – potential source of a wide range of interesting applications – and the algorithms and tools that could be of use to perform specific actuation works. Third, most research work is being devoted to the definition of "power-effective" algorithms, aimed at minimizing resource consumption. This is motivated by the current impossibility of providing such small computer systems with enough battery power to last for a long time. However, we think that short-life batteries and the consequent need of power-aware computing models are a current contingent problem, rather than a basic research issues likely to have long-term impact. Scavenging power from sunlight, vibration, thermal gradients, and background RF, next generation of microcomputers will be fully autonomous in terms of power supply, and will be capable of long-lasting, if not ever-lasting, activity [Pis03].

The Amorphous Computing project at MIT, more closely approaching our vision of spray computers, focuses on the problem of identifying suitable models for programming applications over amorphous networks of computational particles [Nag03]. The particles constituting an amorphous computer have the capabilities of locally propagating sorts of computational fields in the network, and to locally sense and react to such fields (the field abstraction has been reverse-engineered from the biological concept of morphogen gradients). By having particles sense and re-propagate these fields, coordinated patterns of activities emerge in the system independently of the structure of the network. Moreover, the Amorphous computing project has defined a simple yet effective language for programming particles on the basis of such computational fields. It has been shown how it is possible to exploit such a language to let the particles (directly) self-organize a coordinate systems and self-determine their position in it, and to have a variety of global patterns getting (directly) organized in a system from local interactions. Areas that the project has still not properly addressed concerns: the extensive reverse engineering exploitation of emergent phenomena and the study of networks with mobile and ephemeral particles.

Explicitly focused on mobility are those work on in the area of cooperative mobile robotics [McL04, Pay01, MamVZ04, StoN04], which also pay more attention to emergent phenomena. These works, as in the examples reported in the previous section, try to reverse engineer natural phenomena such as chemical diffusion and embryogenesis activities to drive the spatial assembling

and the movements of a swarm of tiny mobile robots (particles). In these works mobile robots connect with each other in an ad-hoc network to coordinate their movements. The main reversed abstractions being researched are those of chemical-gradients diffusion (that can be some somewhat assimilated to the computational fields exploited in the Amorphous Computing Project) and local density estimation. These abstractions can be used by the robots to coordinate their movements. In particular, gradients can drive robots to reach specific locations [Pay01, MamVZ04]. Density can be used to create diffusion-like processes, where robots tend to stay as far as possible from each other [McL04]. Both these two abstractions can be reverse-engineered in terms of distributed data structures (e.g., field-like hop-increasing structures and gradients of density pressures) over the ad-hoc network defined by robot themselves. Despite some promising results, up to now, the cited works focused on very simple motion strategies and computational particles – not much different from cellular automata cells. Their effectiveness in coordinating systems of more complex particles and in more complex application scenarios is still to be verified.

Very similar considerations can apply to the area of motion gait enforcement in modular robots [SheS02, Polybot] and to the area of motion coordination for unmanned autonomous vehicles [Par04].

### 4.2 Medium-scale

Coming the medium scale, as far as we can see most of the researches are focusing either on routing algorithms for mobile ad-hoc networks of handheld computers [Bro98, Kuh03] or on the definition of effective user-level ubiquitous environments [Rom02, Wan04].

Researches on routing algorithms for mobile networks share several common issues with researches on algorithms for data distribution on sensor networks, and often are based on self-localization and spatial concepts [Kuh03], i.e., relying on direct engineering approaches. In our opinion, these works are, again, too often focused on power and resources limitation problems and mostly disregard higher-level self-organization issues such as coordination of distributed behaviors.

Researches on ubiquitous computing environments mostly focus on achieving dynamic interoperability of existing application-level components and of smart-artifact and pervasive computing devices. For instance, the Gaia system developed at PARC [Rom02], defines an architecture based on "active" interaction spaces, as a reification of a specific real-world environment (e.g., a meeting room), where pre-existing (and pre-programmed) devices and user-level software components can dynamically enter, leave and interoperate dynamically with each other according to specific patterns specified as part of the active environment. Although such an approach is very important to organize user-level activities and their interactions with a smart environment, neither Gaia nor most of the other proposals in this direction has something to say on the issue of designing, developing, and controlling self-organizing coordinated distributed applications.

The specific current research area of the first and third authors goes exactly in the direction of extensively applying self-organization mechanisms and phenomena in the context of pervasive computing. The idea of computational fields (or morphogen gradients) has been applied in the implementation of the TOTA middleware [MamZ04]. TOTA relies on spatially distributed tuples, implementing the concept of computational field, for adaptive and context-aware interactions. Application components can inject these tuples in the network, to make available some kind of contextual information and to interact with other agents. Tuples are propagated by the middleware in the pervasive network, on the basis of application specific patterns, defining sorts of "computational fields", and their intended shape is maintained despite network dynamics, such as topological reconfigurations. Agents can locally "sense" these fields and can rely on them for both acquiring contextual information and carrying on distributed self-organizing coordination activities. The TOTA model is effective in realizing applications centered on "spatial" coordination, where the goal is to coordinate the activities of components in the space – either physical or network-based (e.g. motion coordination, self-assembly, network routing, etc.) – and can be effectively used in a general-purpose way to program and deploy applications exploiting both direct reverse self-organization approaches. However, the generality of this approach in supporting very large-scale applications for highly dynamic networks, as well as in deploying mixed approaches for the control of self-organization, is still to be proved.

As an additional research direction, the idea of self-organizing pheromone-based coordination, as inspired by ant-foraging [BonDT99, Par97], is being currently experienced by having the components of a pervasive environment deposit pheromones in the form of RF-ID tags [MamZ05].

### 4.3 Global-scale

As far as the global scale is involved, we have already stated that most research on adaptive and unsupervised computing focus, so far, on the key idea of defining structured self-organizing overlay networks for P2P computing. This can be considered as a typical example

of a direct engineering approach to self-organization [Rat01, RowD01, CasCR04].

Most research work on structured P2P overlays is indeed very valuable and has provided several insights on the actual problems involved in self-organized computing at a world-wide scale. However, this does not necessarily imply that structured overlay networks are the best approach to promote self-organizing global scale computing. In fact, building and maintaining globally coherent overlay networks at a worldwide scale may be very costly. Thus, despite the simulation on small-scale systems show the feasibility of the approach, it is not very clear how this could scale to millions of nodes. In addition, although most of the proposals for overlay networks prove their effectiveness in re-organizing a coherent structure upon dynamic changes in the structure, such studies are typically performed by testing the sequential arrival/dismissing of single nodes, and it is not clear if higher degree of networks dynamics (with concurrent arrivals/dismissing of nodes) would be sustained equally well. In our opinions, next generation P2P should better rely on more flexible and light-weight approaches, possibly exploited reverse engineering approaches, or sort of mixed approaches.

In this context, some recent works in the area of Internet routing and distributed interactions root on reverse engineering ideas coming from biology – i.e. ant foraging [BabMM02, BonDT99, BraE02], or physics – i.e. potential fields [MamZL04]. As an example of this class of approaches (based on artificial ants), Anthill [BabMM02] support the design and development of adaptive peer-to-peer applications by relying on distributed mobile components ("ants") that can travel and can indirectly interact and cooperate with each other by leaving and retrieving bunches of information (to act as synthetic pheromones) in the visited hosts. The key objective of anthill is to build robust and adaptive "semi-structured" networks of peer-to-peer services by exploiting the capabilities of ants to re-organize their activity patterns accordingly to the changes in the network structure. As another example, Swarm-Linda [MenT03] (developed by the fourth author) is an ant-inspired system enabling the globally distributed application components to adaptively coordinate with each other. Application components on the Internet can access a set of independent tuple spaces to retrieve and deposit information. Swarm of ant-agents roam the Internet performing a kind of foraging activity that create routes to guide application components in accessing the proper tuple spaces. Although we consider both SwarmLinda and Anthill as very promising approaches, they still lack of general methodologies to engineer and configure the systems to work as desired.

Another promising thread of researches strictly related to self-organization for global scale (and not only) computing is in the multiagent systems community. Multiagent systems researches aims at identifying suitable models and tools to enable the definition of adaptive applications based on autonomous software components, capable of dynamically interacting with each other and with the (computational) environment in which they situates, so to achieve global application goals. For instance, the second author has defined a specific methodology, ADELFE [Ber02], supporting the design and development of self-organizing applications based on cooperative adaptive agents. The methodology, which has been applied among the other to adaptively manage a global workflow application, consider modeling an application in terms of application agents that inspect the behavior of the systems and adaptively change their behavior so as to direct the evolution of a system as required by some global application goal.

## 5 Research Agenda

To conclude, we sketch a rough research agenda for what we believe are the key challenges to be faced in the area of self-organization for the design, development, and control, of spray computer applications.

First, we think that researches should rely on a deeper understanding of the global behavior of spatially distributed systems of autonomous and interacting components, in any area. This could be used to exploit principles of self-organization offensively, as discussed in Section 3.2, i.e., to achieve via reverse engineering adaptive and robust globally coordinated behaviors in a very simple and effective way. While several approaches already take inspiration from a number of natural phenomena (e.g., ant-inspired algorithms and coordination based on computational fields), a number of currently underestimated phenomena occurring in other types of spatially distributed systems of autonomous components (e.g., macro-ecology patterns of population distribution and biodiversity, physics of granular media, emergence of synchronization, morphogenesis) [ShiM02, Str03] are still to be deeply investigated. Such investigations could not only lead to new interesting solutions, but could also illuminate on the potential emergence of possibly dangerous self-organizing behaviors in computational systems prior to their deployment. Clearly, all of this research work should be coupled with extensive simulation practice [EdmB04].

Together with the study of self-organization phenomena, it will be necessary to study which mechanisms and tools can be adopted to somehow direct the behavior such self-organized systems in a decentralized way. This will be necessary to enforce some sorts of control over these systems, to ensure that

they will behave as desired, despite the impossibility of controlling them in their full, and keeping into account that a full "direct" control would undermine the basic advantages of self-organization. Besides the two simple examples reported in this paper, this is a nearly virgin area of research.

An additional general issue that is to be outlined is that – in several cases – the reverse engineering of some natural phenomena may lead to nothing but to *solutions looking for problems*. In these years where biologically and socially inspired solutions appears to "sell well" in the scientific community, the risk is that of forgetting real-world problems to focus only on problems for which nice biologically-inspired solutions can be found, developed by reverse engineering of some natural phenomenon. Obviously, good scientific and engineering practice recall us that problems come first, and that in most of the cases it is difficult to find a ready-to-use solution for a new problem. To solve the dilemma, one should recall that computational systems, unlike natural systems, do not obey to physical laws. Rather, they live in an environment for which programmers can be the creator of any desired virtual physical law. Thus, researchers are by no means limited to find out useful applications for existing self-organization phenomena but could, in principle, invent new mechanisms and laws, leading to new self-organization phenomena specifically suited to specific application problems. While only a few visionary people will be able to "see" what this new phenomena could be and how they could be made emerge, most of us will probably have to exploit evolutionary approaches (e.g., genetic algorithms or learning agents) to generate them in an empirical way, and then reverse engineering them as we are already used to with natural phenomena [Jac01, Ken01]. In this context, it is worth outlining that some approaches exploiting learning theories and evolutionary approaches to have a system autonomously learn how to self-organize itself have already been proposed, e.g., in the area of sensor networks [Cat02, Sim03] and of multiagent systems [Ber02].

Once the above understanding will be quite assessed, we think there will be need to define a general purpose programming model for designing and deploying applications in such dynamic networks of spray computers, together with the development of associated middleware infrastructure and tools. One very ambitious objective could be for such a model to enable people to program, deploy, and control self-organizing and adaptive distributed applications (exploiting both direct and reverse engineering approaches) with a minimal background knowledge – the same as a undergraduate students can currently develop excellent distributed Web-based Java applications – and independently of the specific application scenario, sensor networks rather than wide-area distributed applications – the same as an undergraduate student can easily and with minimal efforts adapt its applications for execution on both a Linux workstation and a Cellular phone. The definition of such a model will clearly require the identification of a minimal set of abstractions enabling the modeling of salient characteristics of spray computers and their operational environments. In our opinion, approaches based on computational fields [MamVZ04, MamZ04], which have been mentioned several times in previous sections, are very promising to this purpose, by enabling to uniformly model a wide variety of distributed self-organizing behaviors (both with direct and reverse engineering) and to effectively model also ant-inspired approaches [MenT03]. However, this opinion is still to be verified.

Eventually, all the above researches will definitely increase our understanding on the potentials of spray computers at any scale, and will likely cause a range of new application areas to come to the fore. For instance, systems such as worldwide file sharing and artifacts like the cloak of invisibility could have simply never been conceived a few years ago. The new software and hardware technology will call also for visionary application-oriented thinkers, to unfold in full the newly achieved application potentials.

As a final note, we also want to emphasize that the widespread and pervasive diffusion of self-organizing distributed computing systems to which we will assist in the next few years will not come without dangers. Even without referring to the (scientifically improbable) catastrophic scenarios depicted by Michael Chricton, more pragmatic problems will have to be faced such as pollution due to (literally) spray computers being dispersed in the environment and garbage collection of obsolete spray computer software.

## References


[AlbJB00] R. Albert, H. Jeong, A. Barabasi, "Error and Attack Tolerance of Complex Networks", *Nature*, 406:378-382, 27 July 2000.

[BabMM02] O. Babaoglu, H. Meling, A. Montresor, "Anthill: A Framework for the Development of Agent-Based Peer-to-Peer Systems", *22nd International Conference on Distributed Computing Systems*, IEEE CS Press, Vienna (A), June 2002.

[BerG97] A. A. Berlin, K. J. Gabriel, "Distributed MEMS: New Challenges for Computation", *IEEE Computing in Science and Engineering*, (4)1:12-16, Jan.-March. 1997.

[Ber02] C. Bernon, M.P. Gleizes, S. Peyruqueou, G. Picard, "ADELFE: a Methodology for Adaptive Multi-Agent Systems Engineering", *3rd International Workshop Engineering Societies in the Agents World*, LNAI No. 2577, Sept. 2002.

[BonDT99] E. Bonabeau, M. Dorigo, G. Theraulaz, *Swarm Intelligence*, Oxford University Press, 1999.

[BraE02] D. Braginsky, D. Estrin, "Rumor Routing Algorithm For

Sensor Networks", *1st Workshop on Sensor Networks and Applications (WSNA)*, Sept. 2002.

[Bro98] J. Broch, D. Maltz, D. Johnson, Y. Hu, J. Jetcheva, "A Perfomance Comparison of Multi-Hop Wireless Ad Hoc Network Routing Protocols", *ACM/IEEE Conference on Mobile Computing and Networking*, ACM Press, Dallas (TX), Oct. 1998.

[CasCR04] M. Castro, M. Costa, A. Rowstron, "Performance and Dependability of Structured Peer-to-Peer Overlays", *International Conference on Dependable Systems and Networks*, IEEE CS Press, Firenze (I), July 2004.

[Cat02] E. Catterall, K. Van Laerhoven, M. Strohbach, "Self-Organization in Ad-Hoc Sensor Networks: An Empirical Study". *Artificial Life VIII*, Sydney (AU), MIT Press, 2002.

[Cri02] M. Crichton, *Prey: a Novel*, Harper & Collins, 2002.

[EdmB04] B. Edmonds, J. Bryson, "The Insufficiency of Formal Design Methods - The Necessity of an Experimental Approach for the Understanding and Control of Complex MAS", *3rd International Conference on Autonomous Agents and Multiagent Systems*, New York (NY), July 2004.

[Est02] D. Estrin, D. Culler, K. Pister, G. Sukjatme, "Connecting the Physical World with Pervasive Networks", *IEEE Pervasive Computing*, 1(1):59-69, Jan. 2002.

[GelC92] D. Gelernter, N. Carriero, "Coordination Languages and Their Significance", *Communications of the ACM*, (35)2:96-107, February.

[HigB01] J. Hightower, G. Borriello, "Location Systems for Ubiquitous Computing", *IEEE Computer*, 34(8):57-66, Aug. 2001.

[ImiG00] T. Imielinski, S. Goel, "Dataspace - Querying and Monitoring Deeply Networked Collections in Physical Space", *IEEE Personal Communications Magazine*, October 2000, pp. 4-9.

[Jac01] C. Jacob, *Illustrating Evolutionary Computation with Mathematica*, Morgan Kauffman Publisher, San Francisco, 2001.

[Ken01] J. Kennedy, R. Eberhart, *Swarm Intelligence*, Morgan Kauffman Publisher, San Francisco, 2001.

[KepC03] J. Kephart, D. M. Chess, "The Vision of Autonomic Computing", *IEEE Computer*, 36(1):41-50, Jan. 2003.

[Kuh03] F. Kuhn, R. Wattenhofer, Y. Zhang, A. Zollinger, "Geometric Ad-Hoc Routing: Of Theory and Practice", *22nd ACM Symposium on Principles of Distributed Computing*, ACM Press, Boston (MA), July 2003.

[MamRZ05] M. Mamei, A. Roli, F. Zambonelli, "Emergence and Control of Macro Spatial Structures in Perturbed Cellular Automata, and Implications for Pervasive Computing Systems", *IEEE Transactions on Systems, Man, and Cybernetics*, 36(5), May 2005, to appear.

[MamVZ04] M. Mamei, M. Vasirani, F. Zambonelli, "Experiments of Morphogenesis in Swarms of Simple Mobile Robots", *Journal of Applied Artificial Intelligence*, 8(9-10):903-919, October 2004.

[MamZ04] M. Mamei, F. Zambonelli, "Programming Pervasive and Mobile Computing Applications with the TOTA Middleware", *2nd IEEE Conference on Pervasive Computing and Communications*, Orlando (FL), IEEE CS Press, March 2004.

[MamZL04] M. Mamei, F. Zambonelli, L. Leonardi, "Co-Fields: a Physically Inspired Approach to Distributed Motion Coordination", *IEEE Pervasive Computing*, 3(2):52-60, April 2004.

[MamZ05] M. Mamei, F. Zambonelli, "Physical Deployment of Digital Pheromones Through RFID Technology", *Technical Report No. DISMI-UNIMORE-001*, Università di Modena e Reggio Emilia, January 2005.

[McL04] J. McLurkin, J. Smith, "Distributed Algorithms for Dispersion in Indoor Environments using a Swarm of Autonomous Mobile Robots", *7th International Symposium on Distributed Autonomous Robotic Systems*, Toulouse (F), 2004.

[MenT03] R. Menezes, R. Tolksdorf, "SwarmLinda: a New Approach to Scalable Linda Systems based on Swarms", *16th ACM Symposium on Applied Computing*, Melbourne (FL), March 2003.

[Nag03] R. Nagpal, A. Kondacs, C. Chang, "Programming Methodology for Biologically-Inspired Self-Assembling Systems", *AAAI Spring Symposium on Computational Synthesis*, March 2003.

[Par97] V. Parunak, "Go to the Ant: Engineering Principles from Natural Agent Systems", *Annals of Operations Research*, 75:69-101, 1997.

[Par04] V. Parunak, S. Brueckner, J. Sauter, "Digital Pheromones for Coordination of Unmanned Vehicles", *1st AAMAS Workshop on Environments for Multi-agent Systems*, LNAI 3374, Springer Verlag, 2004.

[ParBS02] V. Parunak, S. Bruekner, J. Sauter, "ERIM's Approach to Fine-Grained Agents", *NASA/JPL Workshop on Radical Agent Concepts,* Greenbelt (MD), Jan. 2002.

[Pat03] D. Patterson, L. Liao, D. Fox, H. Kautz, "Inferring high-level behavior from low-level sensors", *UBICOMP*, Seattle (WA), October 2003.

[Pay01] D. Payton, M. Daily, R. Estowski, M. Howard, C. Lee, "Pheromone Robotics", *Autonoumous Robots*, 11(3):319-324, 2001.

[Pis00] K. Pister, "On the Limits and Applicability of MEMS Technology", *Defense Science Study Group Report*, Institute for Defense Analysis, Alexandria (VA), 2000.

[Pis03] K. Pister, Invited Plenary Talk, *23rd International Conference on Distributed Computing Systems*, Providence (RI), May 2003.

[Polybot] Modular Reconfigurable Robotics at PARC, http://www2.parc.com/spl/projects/modrobots.

[Rat01] S. Ratsanamy,, P. Francis, M. Handley, R. Karp, "A Scalable Content-Addressable Network", *ACM SIGCOMM Conference 2001*, Aug. 2001.

[RipIF02] M. Ripeani, A. Iamnitchi, I. Foster, "Mapping the Gnutella Network", *IEEE Internet Computing*, 6(1):50-57, Jan.-Feb. 2002.

[Rom02] M. Roman et al., " Gaia : A Middleware Infrastructure for Active Spaces", *IEEE Pervasive Computing*, 1(4):74-83, Oct.-Dec. 2002.

[RowD01] A. Rowstron, P. Druschel, "Pastry: Scalable, Decentralized Object Location and Routing for Large-Scale Peer-to-Peer Systems", *18th ACM Conference on Middleware*, Heidelberg (D), Nov. 2001.

[ShiM02] T. Shinbrot, F. J. Muzzio, "From Noise to Order", *Nature*, 410:251-258, March 2001.

[Sim03] S. Simic, "A Learning-Theory Approach to Sensor Network", *IEEE Pervasive Computing*, 2(4):44-49, 2003.

[StoN04] K. Stoy, R. Nagpal, "Self-Reconfiguration Using Directed Growth", *7th International Symposium on Distributed Autonomous Robotic Systems*, Toulouse (F), 2004.

[Str03] S.Strogatz, *Synchrony*, Theia Publishing, 2003.

[Ten00] D.Tennenhouse, "Proactive Computing", *Communications of the ACM*, 43(5):43-50, May 2000.

[Wan04] X. Wang, J. Song Dong, C. Chin, S. Hettiarachchi, D. Zhang,

"Semantic Space: an Infrastructure for Smart Spaces", *IEEE Pervasive Computing*, 3(3):32-39, July 2004.

[ZamM02] F. Zambonelli, M. Mamei, "The Cloak of Invisibility: Challenges and Applications", *IEEE Pervasive Computing*, 1(4):62-70, Oct.-Dec. 2002.

[ZamM05] F. Zambonelli, M. Mamei, "Spatial Computing: an Emerging Paradigm for Autonomic Computing and Communication", *1st International Workshop on Autonomic Communication*, to appear in LNCS, 2005.